University of Westminster

7COSC013W.1 Foundations of AI
Coursework 1 – Report

Crime-Aware Pedestrian Route Planning

A Hybrid AI System Combining Machine Learning and Search Algorithms for Safe Navigation

Category 3: Hybrid Approaches (Search + Learning)

Academic Year: 2025/26

Submission by: Aswanath Jayanath Sumi
W122660070

ABSTRACT

This report presents the design and implementation of a crime-aware pedestrian route planning system for the Central London. This system is a hybrid artificial intelligence approach, combining both supervised machine learning techniques with graph-based search algorithms to generate routes that balance travel distance with personal safety. Using over 1 million crime records from the Metropolitan Police Service, the system trains classification models (Logistic Regression and Random Forest) to predict area risk levels based on crime severity features, which are then integrated as edge weights in a road network graph containing 69,863 nodes and 182,370 edges.

The machine learning models use two severity-based features (avg_severity and max_severity) to predict high-risk areas. Logistic Regression achieved ROC-AUC of 0.9701 with 95.76% recall for high-risk areas, whilst Random Forest achieved higher overall ROC-AUC (0.9922) but lower recall (92.71%). Due to the safety critical nature of the application where false negatives (missing high-risk areas) are more harmful, Logistic Regression was selected as the primary model.

Experimental evaluation on a test route from Westminster Abbey to Angel Station demonstrates that the hybrid approach achieves a 9.38% reduction in route risk with only a 0.49% increase in distance (+24 meters), whilst A* provides a 3.78x computational speed over Dijkstra's algorithm. The system successfully addresses the Category 3 (Hybrid Approaches) coursework requirement by meaningfully integrating search and learning techniques.

Table of Contents

1.Introduction

Urban pedestrian safety represents a significant concern in modern city planning and navigation systems. Traditional route planning applications focus exclusively on minimizing travel time or distance, it dose not consider critical dimension of personal safety. This oversight is especially troubling in the big cities where crime rates can differ a lot from one street to another and between neighborhoods. Creating navigation systems that consider safety of a person while keeping routes efficient is a perfect example of where hybrid artificial intelligence methods can be applied.

This project tackles this issue by creating a crime aware route planning system that merges machine learning methods with traditional search algorithms. This will fit this course work into Category 3 (Hybrid Approaches) as outlined in the coursework requirements, focusing on a “Search + Learning” part where risk predictions from learning influence the heuristics and edge weights applied by search algorithms.

1.1 Project Objective

1.first I Developed a machine learning pipeline to predict crime risk levels from crime data.

2.Implement and compare multiple search algorithms (Dijkstra and A*) for route optimisation.

3.Integrate ML predictions with search algorithms to create a hybrid AI system.

4.Also consider environmental safety factors based on CPTED principles.

5.Evaluate the what I achieved and compromised between route distance and safety

6.Compare and analyze two AI approaches

1.2 Learning Outcomes Addressed

This project addresses the following learning outcomes:

- Exploration Methods:Utilizing Dijkstra's algorithm and A* search for route planning, showcasing how these techniques are adjusted to include safety measures by altering edge weights.

- Learning Techniques:Applying supervised learning techniques (Logistic Regression and Random Forest) to predict crime risk, with assessment of model performance, feature selection, and the precision-recall trade-off.

- AI Strategy Recommendation: Critically assessing and recommending the hybrid AI strategy, justifying the combination of search and learning approaches and model selection based on application requirements.

2. Problem Statement and Real-World Context

2.1 Problem Definition

The problem addressed in this coursework is crime-aware pedestrian route planning in an urban environment. Given a start location and a destination in Central London, the aim is to compute a walking route that balances two competing objectives:

- Route efficiency (minimizing total travel distance)
- Personal safety (minimizing exposure to historically higher-crime areas and lower-surveillance streets).

Unlike traditional navigation systems that focus only for time or distance, this project explicitly treats safety as an optimisation objective and integrates it with the routing process.

This can be expressed as a multi-objective optimisation problem where the total path cost is a weighted combination of distance and risk:

cost(path)= (1 - alpha) * distance + alpha * risk * distance

Where:

- Distance(path) is the sum of road segment lengths in meters along the path,
- Risk(path) is the total risk score across segments (derived from crime risk predictions and environmental modifiers)
- Alpha is a element of [0, 1] controls the trade-off between efficiency and safety.
    - alpha~0 yields a conventional shortest-distance route
    - alpha~1 yields a safety maximized route regardless of distance

In this coursework, alpha = 0.7 is used to prioritize safety while maintaining practical route efficiency.

2.2 Real-World Context and Motivation

Pedestrian safety is a practical concern in large cities due to substantial spatial variation in crime patterns.According to Metropolitan Police Service statistics, over 1.1 million street-level crimes were recorded in the greater London area during the 2023-2024 reporting period. These crimes range from low-severity incidents such as anti-social behavior to high-severity offenses including violence and robbery.

Current navigation systems like google map, city mapper priorities travel time or distance and do not incorporate safety as a routing criterion. This creates a gap where users may be directed through areas that are efficient but riskier based on historical crime incidents.

This System can be relevant for users prefer:

- Late-night walking routes.
- Tourists unfamiliar with local area risk profiles.
- Individuals who prefer safer and more visible streets even at the cost of slight detours.

2.3 Why a Hybrid AI Approach?

A hybrid AI approach is appropriate for our project, because the problem requires both:

- Learning from data to estimate risk, and
- Search/optimisation to compute an optimal route over a road network.

Search alone is insufficient:

Graph search algorithms (e.g, Dijkstra, A*) can compute optimal paths, but they require meaningful edge weights. If risk is not modeled from data, the system would rely on simplistic heuristics or raw crime counts that may not reflect real severity patterns.

Machine learning alone is insufficient:

Machine learning can predict the relative risk of spatial areas from historical data, but it does not solve the combinatorial optimisation problem of finding the best sequence of connected road segments between two points.

Synergistic integration:

This project integrates both approaches: the machine learning component predicts risk levels for spatial areas using crime severity features, and those predictions are then used to define edge weights for routing algorithms. This creates a complete, functional hybrid system that directly satisfies the Category 3 (Search + Learning) requirement.

2.4 Success Criteria and Evaluation Metrics

Project success is evaluated using measurable criteria :

Machine Learning (Risk Prediction)

- ROC-AUC to assess ranking/discrimination performance across thresholds
- High-risk recall as a safety-critical metric (minimizing missed high risk areas)
- Precision/Recall trade-off to justify model selection based on application goals

Routing (Search Performance)

- Distance overhead: percentage increase in route length compared to the shortest path
- Risk reduction: percentage decrease in accumulated route risk compared to the distance-only route
- Computational efficiency: runtime comparison between A* and Dijkstra under equivalent cost functions

The core safety-efficiency trade-off is evaluated by comparing:

- the baseline shortest route (distance-only), and
- the hybrid safety-aware route (risk-weighted cost).

A successful outcome is one where the hybrid route demonstrates a meaningful reduction in risk while keeping distance increase small enough to remain practical for pedestrians.

3. Proposed AI Techniques

This section describes the artificial intelligence techniques used in the system. The solution combines supervised machine learning for crime risk prediction with graph-based search algorithms for route optimisation, forming a hybrid AI approach that balances safety and efficiency.

3.1 Machine Learning Techniques

3.1.1 Feature Selection Rationale

A key design decision in this project was the selection of appropriate features for the machine learning models. After spatial aggregation of crime data, three candidate features were selected:

- avg_severity – the mean crime severity within each spatial grid cell
- max_severity – the maximum observed crime severity in each cell
- crime_count – the total number of crimes in each cell

The target variable, high_risk, was defined using a threshold on crime_count (high_risk = 1 if crime_count exceeds the 60th percentile).

```
crime_density_threshold = risk_df['crime_count'].quantile(0.60)

risk_df['high_risk'] = (risk_df['crime_count'] > crime_density_threshold).astype(int)
```

Including crime_count as an input feature would therefore introduce target leakage, allowing the model to trivially learn the threshold rule rather than meaningful risk patterns.

To avoid this issue, crime_count was explicitly excluded from the feature set. The final features used for prediction are:

- avg_severity: captures the typical level of crime severity in an area
- max_severity: captures worst-case crime severity, which is particularly relevant for pedestrian safety

Using these two severity-based features creates a genuine prediction task in which the model must learn how crime severity patterns relate to area risk, rather than memorizing a simple count-based rule.

The original crime dataset contains multiple attributes, including Crime ID, Month, Reported by, Falls within, Longitude, Latitude, Location, LSOA code, LSOA name, Crime type, Last outcome category, and Context. Many of these fields are either identifiers, textual descriptors, or administrative metadata and therefore do

not provide direct predictive value for modelling spatial crime risk.

```
Columns:
['Crime ID', 'Month', 'Reported by', 'Falls within', 'Longitude', 'Latitude', 'Location', 'LSOA code', 'LSOA name', 'Crime type', 'Last outcome category', 'Context']
```

At the beginning of the project, I concentrated on numerical and spatially relevant data that could be reliably combined at the grid-cell level. We used geographic coordinates, like latitude and longitude, only for spatial aggregation, leaving out temporal and identifier fields such as Crime ID, Reported by, and LSOA fields, as they don't help with risk prediction after aggregation.

Even though crime type is a categorical feature that could be encoded using methods like one-hot encoding, I decided not to include it in the machine learning feature set. Instead, I used a severity mapping strategy to incorporate crime severity information indirectly. Each road type was given a severity score of low, medium, or high. Including both one-hot encoded crime types and severity scores would be unnecessary and might lead to double-counting the similar data.

This design choice emphasizes model simplicity, interpretability, and consistency, while still capturing the essential safety-related information needed for risk prediction. Although more complex encodings could be explored in future work, this approach supports the project's goal of creating a strong and explainable hybrid AI system.

3.1.2 Logistic Regression

Logistic Regression was selected as the primary model because the task is a binary classification problem, where the objective is to predict whether a spatial cell is high-risk or low-risk. The algorithm directly models class probabilities using a sigmoid

function, making it well-suited for safety-critical decision-making and risk-aware routing.

The model was configured with:

- class_weight = 'balanced' to address mild class imbalance,
- max_iter = 1000 to ensure convergence, and
- random_state = 42 for reproducibility.

3.1.3 Random Forest Classifier

A Random Forest classifier was implemented as a second model to provide a comparative baseline. It can model non-linear relationships between severity features and risk

The model was configured with 100 decision trees, a maximum depth of 10, balanced class weights, and a fixed random seed for reproducibility.

3.1.4 Model Selection: Precision–Recall Trade-off

Evaluation revealed a clear precision–recall trade-off between the two models, as shown in Table 3.1.

Metric	Logistic Regression	Random Forest
Precision (High-Risk)	90.44%	99.49%
Recall (High-Risk)	95.76%	92.71%
ROC-AUC	0.9701	0.9922

Table 3.1: Precision–Recall Trade-off Between Models

Although Random Forest achieves higher overall ROC-AUC and precision, Logistic Regression achieves higher recall for high-risk areas. In a safety-critical routing application, false negatives (misclassifying a genuinely high-risk area as low-risk) are more

harmful than false positives, as they may lead pedestrians into dangerous locations. A false positive, by contrast, typically results in a slightly longer but safer route.

Random Forest is typically advantageous for high-dimensional datasets with many features, where its ensemble approach provides automatic feature selection and captures complex interactions. With only two features (avg_severity, max_severity), the problem is relatively simple, Although Random Forest achieves higher overall ROC-AUC (0.9922 vs 0.9701) and precision (99.49% vs 90.44%), Logistic Regression achieves higher recall for high-risk areas (95.76% vs 92.71%). In a safety-critical routing application, false negatives (misclassifying a genuinely high-risk area as low-risk) are more harmful than false positives, as they may lead pedestrians into dangerous locations. A false positive, by contrast, typically results in a slightly longer but safer route.

For this reason, Logistic Regression was selected as the primary risk prediction model, while Random Forest is retained as a comparative benchmark.

3.2 Search Algorithms and Environmental Safety Integration

This project uses graph-based search algorithms to compute pedestrian routes on a road network, while incorporating safety considerations through both machine learning predictions and environmental design principles. Two classical search algorithms Dijkstra's algorithm and A* search are used and compared.

Dijkstra's Algorithm

Dijkstra's algorithm finds the optimal path in a weighted graph by expanding nodes in order of increasing cost. It guarantees the shortest path when all edge weights are non-negative, making it a reliable baseline for route planning.

In this project, Dijkstra's algorithm is applied in two ways:

- Distance-only routing, where each road segment is weighted by its physical length
- Safety-aware routing, where edge weights combine distance with predicted crime risk

A* Search Algorithm

A* search (Hart, Nilsson, & Raphael, 1968) builds on Dijkstra's algorithm by adding a heuristic that estimates the remaining distance to the destination. This allows the search to focus on more promising paths and significantly reduces unnecessary exploration.

The algorithm evaluates nodes using:

$$f(n)=g(n)+h(n)$$
$$f(n) = g(n) + h(n)$$
$$f(n)=g(n)+h(n)$$

where g(n) is the accumulated path cost and h(n) is a heuristic estimate of the remaining distance.

The heuristic used is the great-circle distance, which provides a straight-line estimate between two locations. This heuristic is admissible and consistent, meaning A* produces the same optimal routes as Dijkstra's algorithm but with improved efficiency.

CPTED-Based Environmental Safety Factors

In addition to machine learning–based crime risk predictions, the system incorporates environmental safety considerations based on Crime Prevention Through Environmental Design (CPTED) principles. Specifically, the concept of natural surveillance is applied: streets with higher visibility and pedestrian activity are generally safer.

Road types are assigned safety modifiers reflecting their typical visibility and usage. For example, primary roads are considered safer due to high activity and lighting, while alleys and isolated footpaths receive higher risk weights. These modifiers do not replace crime data; instead, they adjust the risk score to reflect environmental context.

The final edge risk used for routing combines both components:

```
ROAD_TYPE_RISK = {

    'primary': 0.5, 'primary_link': 0.5,
    'secondary': 0.6, 'secondary_link': 0.6,
    'tertiary': 0.7, 'tertiary_link': 0.7,

    'residential': 0.9,
    'living_street': 1.0,

    'service': 1.3,
    'alley': 1.5,

    'pedestrian': 1.0,
    'footway': 1.1,
    'path': 1.4,
    'steps': 1.3,

    'unclassified': 1.0,
    'road': 1.0
}
```

```
combined_risk = (CRIME_WEIGHT * crime_risk) + (ROAD_TYPE_WEIGHT * normalized_road_risk)
```

combined_risk=0.7×ML-predicted crime risk+0.3×road-type risk

This combined risk is then integrated into the search algorithms, allowing both Dijkstra and A* to generate routes that balance safety and distance.

4. Dataset Description

In this section I describes the crime dataset used in the project, including its source, key characteristics, data quality checks, and the preprocessing steps applied.

4.1 Data Source

The primary dataset used in this project consists of street level crime records published by the Metropolitan Police Service which I accessed through the data.police.uk open data portal. The dataset is publicly available and updated on a monthly basis, making it suitable for transparent and reproducible academic research.I downloaded this data individually then manually combined into one dataset using a simple python code.This dataset has around 12 month of data.

All records are anonymized and contain no personal identifiers, ensuring ethical use of the data.

4.2 Dataset Characteristics

The dataset covers around 12-month period from October 2023 to September 2024 and includes crime incidents across the Greater London metropolitan area.

Attribute	Value
Total Crime Records (Raw)	1,245,107
Duplicate Records Removed	117,824 (9.46%)
Total Crime Records (After Cleaning)	1,127,283
Coverage	(12 months)
Geographic Coverage	Greater London Area

Spatial Grid Cells	5,334 cells
Crime Type Categories	14
Road Network Nodes	69,863
Road Network Edges	182,370

4.3 Data Quality Assessment and Preprocessing

4.3.1 Missing Value Analysis

Missing Values Summary:

Column	Missing_Count	Missing_Percent
crime id	254641	20.45
month	0	0.00
reported by	0	0.00
falls within	0	0.00
longitude	0	0.00
latitude	0	0.00
location	0	0.00
lsoa code	1	0.00
lsoa name	1	0.00
crime type	0	0.00
last outcome category	254641	20.45
context	1245107	100.00

A missing value analysis was performed on the raw dataset:

The key attributes required for this project latitude, longitude, month, and crime type contain no missing values and therefore required no imputation. The context column was entirely empty and was removed. One record with a missing LSOA code was discarded due to its negligible impact, Even though that column was not required, i came to know about it later.

4.3.2 Duplicate Removal

Duplicate records were identified based on exact row matches. A total of 117,824 duplicate entries (9.46%) were removed. This

resulted in a clean dataset containing 1,127,283 unique crime records.

4.3.3 Data Cleaning Pipeline

The following preprocessing steps were applied:

1. Column names standardized to lowercase
2. Removal of irrelevant or empty columns
3. Elimination of duplicate records
4. Validation of geographic coordinates within UK boundaries
5. Feature engineering for crime severity and spatial aggregation

This pipeline ensured consistency, reliability, and suitability for machine learning.

4.4 Crime Severity Mapping

Crime categories were mapped to severity scores ranging from 1 to 3, based on UK Home Office classifications and their potential impact on pedestrian safety.

Severity	Crime Types	Rationale
3 (High)	Violence and sexual offences, Robbery	Direct physical threat
2 (Medium)	Burglary, Vehicle crime, Theft from person, Criminal damage, Public order, Possession of weapons	Indirect or situational risk
1 (Low)	Shoplifting, Anti-social behaviour, Bicycle theft, Drugs, Other theft, Other crime	Lower immediate risk

4.5 Spatial Grid Aggregation

To create stable units for machine learning, individual crime records were aggregated into spatial grid cells using coordinate snapping:

$$\text{grid\_coordinate} = \text{round}(\text{coordinate} / \text{GRID\_SIZE}) \times \text{GRID\_SIZE}$$

With GRID_SIZE = 0.01 degrees, each cell covers approximately 1.1 km × 0.7 km at London's latitude. This results in 5,334 spatial cells, balancing spatial resolution with data sparsity.

4.6 Feature Engineering and Correlation Analysis

For each spatial grid cell, three candidate features were computed:

- avg_severity – mean severity score of crimes in the cell
- max_severity – maximum severity score observed
- crime_count – total number of crimes

5. Implementation

This section provides a detailed walkthrough of the system implementation, presenting the code, outputs, and decision-making process at each stage. This project follows a modular pipeline architecture progressing from data processing through machine learning to route planning integration.

5.1 System Architecture

The system is implemented as a modular pipeline, where each component performs a clearly defined role and passes structured outputs to the next stage. This design improves reproducibility, interpretability, and extensibility.

The architecture consists of three main modules:

1. Data Processing Module
   Uses the cleaned and aggregated dataset containing spatial grid cells with engineered severity features.
2. Machine Learning Module
   Trains supervised classification models to predict whether a spatial cell is high-risk, based solely on severity-based features.
3. Route Planning Module
   Integrates ML-predicted risk scores into a graph-based search framework (Dijkstra or A*) to compute safe pedestrian routes.

This separation allows learning and search components to be developed, evaluated, and modified independently, while still forming a hybrid AI system.

5.2 Development Environment Setup

5.2.1 Library Imports

The implement importing of all necessary libraries for data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,f1_score, roc_auc_score, confusion_matrix,classification_report)
```

processing, machine learning, and visualisation.

I use pandas for data manipulation, numpy for numerical operations, scikit-learn for ML pipelines and evaluation, matplotlib and seaborn for visualisation.

5.2.2 Project Configuration

The project structure establishes separate directories for organised file management.

```
RAW_DATA = Path("/content/data")           # Where you upload CSV
PROCESSED_DATA = Path("/content/processed") # Cleaned data
RESULTS_DIR = Path("/content/results")      # Output charts

for directory in [RAW_DATA, PROCESSED_DATA, RESULTS_DIR]:
    directory.mkdir(parents=True, exist_ok=True)

# Configuration
CSV_FILENAME = "metropolitan_street_full_year.csv"
RANDOM_STATE = 42
TEST_SIZE = 0.2
print(f"Upload CSV to: {RAW_DATA}/{CSV_FILENAME}")
```

we need a data folder in the google collab, and in this folder we need to upload the crime dataset before executing.

Key configuration parameters include RANDOM_STATE=42 for reproducibility and TEST_SIZE=0.2 for an 80/20 train-test split.

5.3 Data Loading and Preprocessing

```
csv_path = RAW_DATA / CSV_FILENAME

if not csv_path.exists():
    raise FileNotFoundError(f"CSV not found at: {csv_path}")

crime_df = pd.read_csv(csv_path, sep=",", quotechar='"',engine="python", on_bad_lines="skip")

print(f"Dataset shape: {crime_df.shape}")
print(f"\nColumns:")
print(crime_df.columns.tolist())
print(f"\nFirst rows:")
crime_df.head()
```

5.3.1 Loading the Crime Dataset (Step 3)

So here I load the csv file using pandas and converted to data frame ,To get a basic idea of a of the data I use shape function and to list function which gives an output:

```
Dataset shape: (1245107, 12)

Columns:
['Crime ID', 'Month', 'Reported by', 'Falls within', 'Longitude', 'Latitude', 'Location', 'LSOA code', 'LSOA name', 'Crime type', 'Last outcome category', 'Context']

First rows:
```

1 to 5 of 5 entries Filter

index	Crime ID	Month	Reported by	Falls within	Longitude	Latitude	Location	LSOA code	LSOA name	Crime type	Last outcome category	Contex
0	e10ab77c2d3898b8553f7fe567b147213e65898260b6b120796497afa11d11ef	2024-10	Metropolitan Police Service	Metropolitan Police Service	0.857345	51.132024	On or near Sheldwich Close	E01024019	Ashford 008B	Violence and sexual offences	Status update unavailable	Nal
1	NaN	2024-10	Metropolitan Police Service	Metropolitan Police Service	0.134947	51.588063	On or near Mead Grove	E01000027	Barking and Dagenham 001A	Anti-social behaviour	NaN	Nal
2	NaN	2024-10	Metropolitan Police Service	Metropolitan Police Service	0.135924	51.587353	On or near Gibbfield Close	E01000027	Barking and Dagenham 001A	Anti-social behaviour	NaN	Nal
3	NaN	2024-10	Metropolitan Police Service	Metropolitan Police Service	0.142112	51.589389	On or near A1112	E01000027	Barking and Dagenham 001A	Anti-social behaviour	NaN	Nal
4	5dc3e8a2b6a8843de68c614a7cdce2cbcf9e1596b10bba4dc82b8a70d122619f	2024-10	Metropolitan Police Service	Metropolitan Police Service	0.140194	51.582356	On or near Hatch Grove	E01000027	Barking and Dagenham 001A	Burglary	Investigation complete; no suspect identified	Nal

Here we can clearly see that this data set has 1245107 crime record and 12 columns, also I have printed the first few column of the dataset using head function.

Now about the columns, the key column or features needed for this project are the latitude , longitude, crime type and month.And the columns that I removed are the crime ID, reported by, falls within, all this are identifies along with this LSOA code/name, last outcome category and the context all this are irrelevant for this project.

The on_bad_lines="skip" parameter ensures robust data loading by safely ignoring malformed rows (e.g., inconsistent delimiters or corrupted entries) that would otherwise cause the import process to fail.

5.3.2 Column Name Standardization (Step 4)

```
crime_df.columns = crime_df.columns.str.lower().str.strip()
```

So here first I convert all the column names to lower case with the lower function and use strip function to remove the spaces that might exist in the column header.

```
LAT_COL = "latitude"
LON_COL = "longitude"
MONTH_COL = "month"
CRIME_COL = "crime type"

required_cols = [LAT_COL, LON_COL, MONTH_COL, CRIME_COL]
```

I also have given a standard name for the required column which should be more easier for me to access.

5.3.3 Missing Value Analysis (Step 5)

```
missing_summary = pd.DataFrame({
    'Column': crime_df.columns,
    'Missing_Count': crime_df.isnull().sum().values,
    'Missing_Percent': (crime_df.isnull().sum().values / len(crime_df) * 100).round(2)
})

print("Missing Values Summary:")
print(missing_summary.to_string(index=False))

print(f"\nCritical columns ({required_cols}):")
for col in required_cols:
    missing_count = crime_df[col].isnull().sum()
    print(f"  {col}: {missing_count} missing")
```

This is the missing value checking step and I checked the sum of null values for each column and stored it in a dictionary, for

printing.Also I printed the number of missing values for each required columns.

```
Missing Values Summary:
                Column  Missing_Count  Missing_Percent
             crime id         254641            20.45
                month              0             0.00
          reported by              0             0.00
         falls within              0             0.00
            longitude              0             0.00
             latitude              0             0.00
             location              0             0.00
            lsoa code              1             0.00
            lsoa name              1             0.00
           crime type              0             0.00
last outcome category         254641            20.45
              context        1245107           100.00

Critical columns (['latitude', 'longitude', 'month', 'crime type']):
  latitude: 0 missing
  longitude: 0 missing
  month: 0 missing
  crime type: 0 missing
```

So here is the output and we can see there are quite few missing values in the dataset, but the good things is all the required features has no missing values.

Step 5: Analyze missing values

```python
print("DUPLICATE ANALYSIS")

duplicates = crime_df.duplicated().sum()
print(f"Exact duplicate rows: {duplicates:,} ({duplicates/len(crime_df)*100:.2f}%)")

if duplicates > 0:
    crime_df = crime_df.drop_duplicates()
    print(f"Removed {duplicates:,} duplicates")
    print(f"New dataset size: {len(crime_df):,}")
else:
    print(" No duplicates found")
```

Here what did was I created a variable and store the sum of duplicate value in that variable, and also I checked if there is any duplicate then stop that rousing drop_duplicate().

```
DUPLICATE ANALYSIS
Exact duplicate rows: 117,824 (9.46%)
Removed 117,824 duplicates
New dataset size: 1,127,283
```

So here the output shows that there is 117,824 total duplicate rows I.e, 9.46% of the total data set.

After removing the duplicate the new dataset has 1,127,283 rows of data.

After that step I removed the unnecessary columns and now the final dataset has only 4 features compared to 12 features.

```
Columns after removal: ['latitude', 'longitude', 'month', 'crime type']
Dataset shape: (1127283, 4)
```

5.3.5 Geographic Validation (Step 7)

```
UK_LAT_MIN, UK_LAT_MAX = 49.8, 60.9
UK_LON_MIN, UK_LON_MAX = -8.2, 2.0

before = len(crime_df)

crime_df = crime_df[(crime_df[LAT_COL].between(UK_LAT_MIN, UK_LAT_MAX))&(crime_df[LON_COL].between(UK_LON_MIN, UK_LON_MAX))].copy()
```

First I have to check if all crime records come under the uk boundaries:

Latitude range: 49.8°N (southern) to 60.9°N (north)
Longitude range: -8.2°W (west) to 2.0°E (east)

These boundaries are based on the Ordnance Survey National Grid reference system, and are also easily available with a simple google search.

Then after comparison I found that all the data are valid as it is inside the uk boundaries.that is no records were removed.

5.4 Feature Engineering

5.4.1 Crime Severity Mapping (Step 8)

```
crime_df[CRIME_COL] = crime_df[CRIME_COL].str.lower().str.strip()

SEVERITY_MAP = {
    "violence and sexual offences": 3,
    "robbery": 3,
    "burglary": 2,
    "vehicle crime": 2,
    "criminal damage and arson": 2,
    "theft from the person": 2,
    "public order": 2,
    "anti-social behaviour": 1,
    "shoplifting": 1,
    "other theft": 1,
    "drugs": 1,
    "bicycle theft": 1,
    "possession of weapons": 1,
    "other crime": 1
}

crime_df["severity"] = crime_df[CRIME_COL].map(SEVERITY_MAP).fillna(1).astype(float)

print(crime_df['severity'].value_counts().sort_index())
```

Here what I did was I mapped the crime type to numeric severity score(1-3) based on there potential impact on the pedestrian safety.

First I convert everything to lowercase,The severity mapping converts categorical crime types into numerical values that represent risk to pedestrians:

For that I map the names in the crime column the SEVERITY_MAP dictionary we just created and this will map

```
crime_df["severity"]
```

	severity
0	3.0
1	1.0
2	1.0
3	1.0
4	2.0
...	...
1245102	3.0
1245103	3.0
1245104	3.0
1245105	3.0
1245106	3.0

1127283 rows × 1 columns

each crime type to a numeric score and the map function replace every category with its mapped number .

Here we can see the output clearly.Aslo if any crime type incase doesn't found any security map then I also added a default value ‘1’, using fullna() function, treating unknown crimes conservatively as low-severity rather than excluding them.for this part I reference pandas data frame fillna page, and also I used Chat-GPT to help me understand and develop this part of code.all the referred web pages will be listed at the end of this document.

Here I could have used one hot encoding to convert this into a numeric column but I find this more customizable I can insert Environmental risk factors modeled separately using CPTED-based road type factors.

5.4.2 Spatial Grid Aggregation (Step 9)

```
GRID_SIZE = 0.01

crime_df['grid_lat'] = (crime_df[LAT_COL] / GRID_SIZE).round() * GRID_SIZE

crime_df['grid_lon'] = (crime_df[LON_COL] / GRID_SIZE).round() * GRID_SIZE

crime_df['cell_id'] = (crime_df['grid_lat'].round(2).astype(str) + '_' + crime_df['grid_lon'].round(2).astype(str))

print(f"Grid size: {GRID_SIZE} degrees")
print(f"Unique cells: {crime_df['cell_id'].nunique():,}")
print(f"Average crimes per cell: {len(crime_df) / crime_df['cell_id'].nunique():.1f}")
```

Here I convert the individual data points to area based risk zone.

How it works:

1. Each coordinate is divided by GRID_SIZE (0.01 degrees)
2. The result is rounded to the nearest integer
3. Multiplying back by GRID_SIZE "snaps" the coordinate to a grid point

Example calculation:

- Original latitude: 51.507432
- Divided by 0.01: 5150.7432
- Rounded: 5151
- Multiplied by 0.01: 51.51 (grid cell centre)

Grid size justification:

- 0.01 degrees ≈ 1.1km × 0.7km
- This creates neighborhood-sized areas small enough for meaningful risk differentiation, large enough to contain sufficient crime data
- Results in 5,334 unique cells across the Metropolitan Police area

Crime coordinates were aggregated into grid cells using coordinate snapping, a technique where each latitude and longitude is rounded to the nearest grid point using the formula: grid_coord = round(coord / grid_size) * grid_size (Stack Overflow, 2012; Police.uk, 2023). This method is consistent with the location anonymization approach used by UK Police crime data.

In this step I am only setting up the grid cell and the risk per each grid cell will be calculated in next step, as we are using trim points even in a single grid cell there could be multiple crimes of different severity and type so I will solve that in the next step.

5.4.3 Risk Statistics Calculation (Step 10)

```
risk_df = crime_df.groupby(['cell_id', 'grid_lat', 'grid_lon']).agg({'severity': ['mean', 'max', 'count'],LAT_COL: 'first',LON_COL: 'first' }).reset_index()

risk_df.columns = ['cell_id', 'grid_lat', 'grid_lon', 'avg_severity','max_severity', 'crime_count', 'lat', 'lon'] #just renamed it

print(f"Risk cells created: {len(risk_df):,}")
print(f"\nStatistics:")
print(f"  Crime count - Mean: {risk_df['crime_count'].mean():.1f}, " f"Median: {risk_df['crime_count'].median():.0f}")
print(f"  Avg severity - Mean: {risk_df['avg_severity'].mean():.1f}, "f"Median: {risk_df['avg_severity'].median():.2f}")
```

This step aggregates crime incidents at the grid-cell level and creates numerical features that can be used to train a machine-learning model.

First I group numeric locations on the cell with the cell_id then I use agg() to summarize crimes inside each cell for each cell the number of crimes, type, severity could be different. So I summarize all crimes within each cell by computing the total number of crimes, the average crime severity, and the maximum crime severity. This converts multiple individual crime records into a single area-level representation, making the data suitable for training a supervised learning model.

Each row = one spatial grid cell

- Each row contains crime intensity and severity statistics
- The data is now area-level, not incident-level

This is a critical transition from raw data -> ML-ready dataset.

```
Risk cells created: 5,334

Statistics:
  Crime count - Mean: 211.3, Median: 2
  Avg severity - Mean: 2.2, Median: 2.03
```

```
risk_df.head()
```

	cell_id	grid_lat	grid_lon	avg_severity	max_severity	crime_count	lat	lon
0	49.97_-5.21	49.97	-5.21	2.0	2.0	1	49.965532	-5.205904
1	50.08_-5.21	50.08	-5.21	2.0	2.0	1	50.077416	-5.209550
2	50.14_-5.08	50.14	-5.08	3.0	3.0	1	50.139279	-5.082957
3	50.15_-5.07	50.15	-5.07	3.0	3.0	1	50.147085	-5.068830
4	50.16_-5.09	50.16	-5.09	3.0	3.0	1	50.157417	-5.088970

```
print("FEATURE CORRELATION ANALYSIS")
print("-" * 40)

# Select numeric columns for correlation
numeric_cols = ['avg_severity', 'max_severity', 'crime_count']
correlation_matrix = risk_df[numeric_cols].corr()

print("\nCorrelation Matrix:")
print(correlation_matrix.round(3))

# Visualize
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            fmt='.3f', square=True, linewidths=0.5)
plt.title('Feature Correlation Heatmap')
plt.tight_layout()
plt.savefig(RESULTS_DIR / 'correlation_heatmap.png', dpi=300)
plt.show()

print(f"\nSaved: {RESULTS_DIR / 'correlation_heatmap.png'}")
```

So this is how the dataset looks like now.

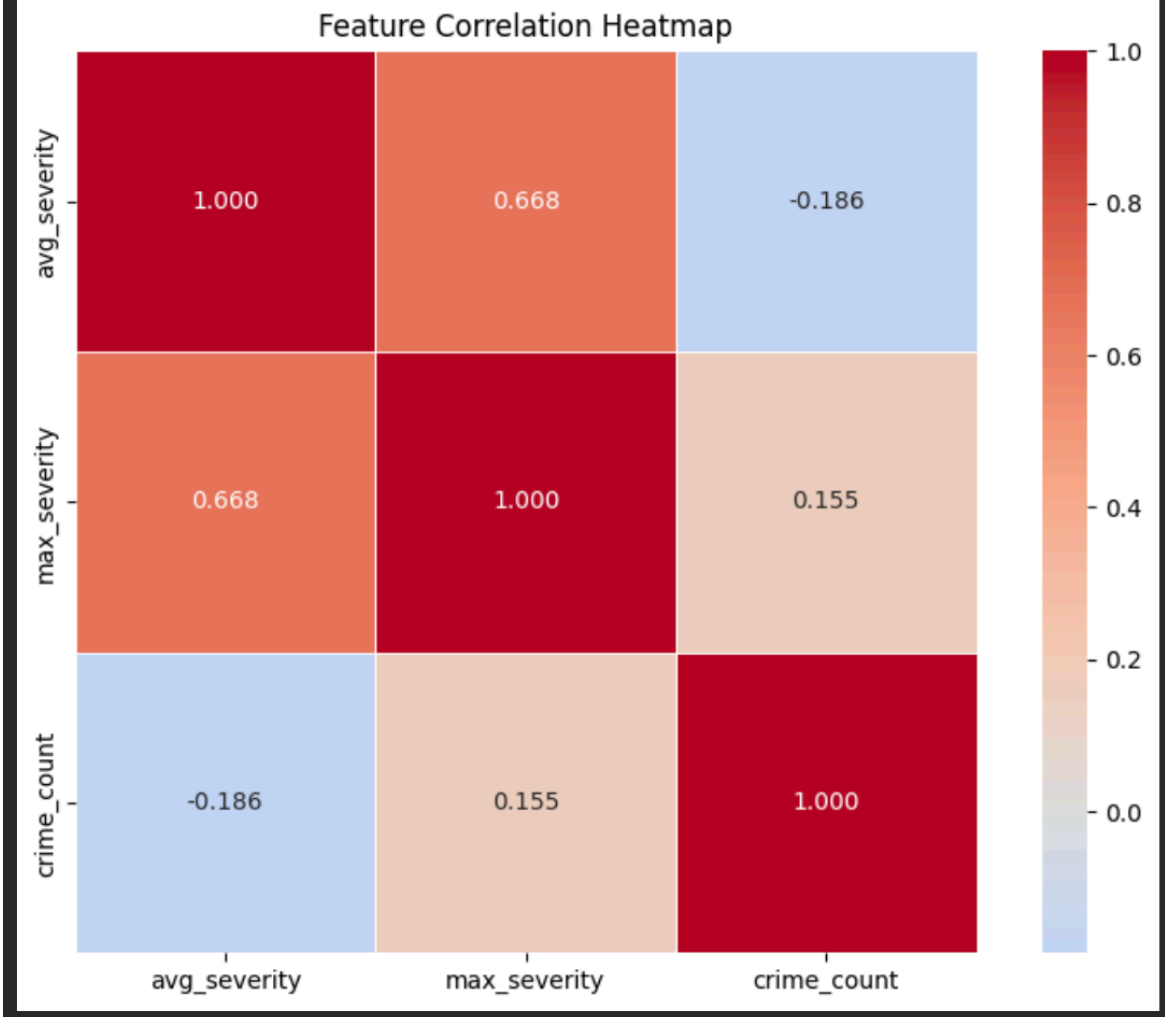
5.4.4 Feature Correlation Analysis (Step 10b)

This will help me understand the relationship between each feature and how much correlated each features are.

The correlation analysis was implemented using pandas and seaborn, with reference to official library documentation.

ChatGPT was additionally used as a conceptual support tool to better understand feature correlation analysis.

```
FEATURE CORRELATION ANALYSIS
----------------------------------------

Correlation Matrix:
              avg_severity  max_severity  crime_count
avg_severity         1.000         0.668       -0.186
max_severity         0.668         1.000        0.155
crime_count         -0.186         0.155        1.000
```



Average severity and maximum severity show a moderate positive correlation (0.668). This means areas with higher average severity usually also have at least one very serious crime. The correlation is not too high, so these features are related but still capture different aspects of risk. Therefore, both were kept.

Average severity and crime count have a weak negative correlation (-0.186). This suggests that areas with many crimes

often contain more low-severity incidents, which lowers the average severity. Since the relationship is weak, these features describe different characteristics of crime risk.

Maximum severity and crime count show a weak positive correlation (0.155). This indicates that areas with more crimes are slightly more likely to include a serious offense, which is expected.Overall, the features are not strongly correlated, so all were retained for model training.

5.5 Machine Learning Pipeline

5.5.1 Target Variable Creation (Step 11)

Here I created the target variable which is basically what the model will try to predict. I used quantile-based thresholding to define which areas are "high-risk".

```
# Step 11: Create binary risk labels
#Define target variable for ML classification

crime_density_threshold = risk_df['crime_count'].quantile(0.60)

risk_df['high_risk'] = (risk_df['crime_count'] > crime_density_threshold).astype(int)

high_risk_count = risk_df['high_risk'].sum()
low_risk_count = len(risk_df) - high_risk_count

print(f"Risk threshold: {crime_density_threshold:.0f} crimes per cell")
print(f"\nClass distribution:")
print(f"  Low risk (0): {low_risk_count:,} cells ({low_risk_count/len(risk_df)*100:.1f}%)")
print(f"  High risk (1): {high_risk_count:,} cells ({high_risk_count/len(risk_df)*100:.1f}%)")
```

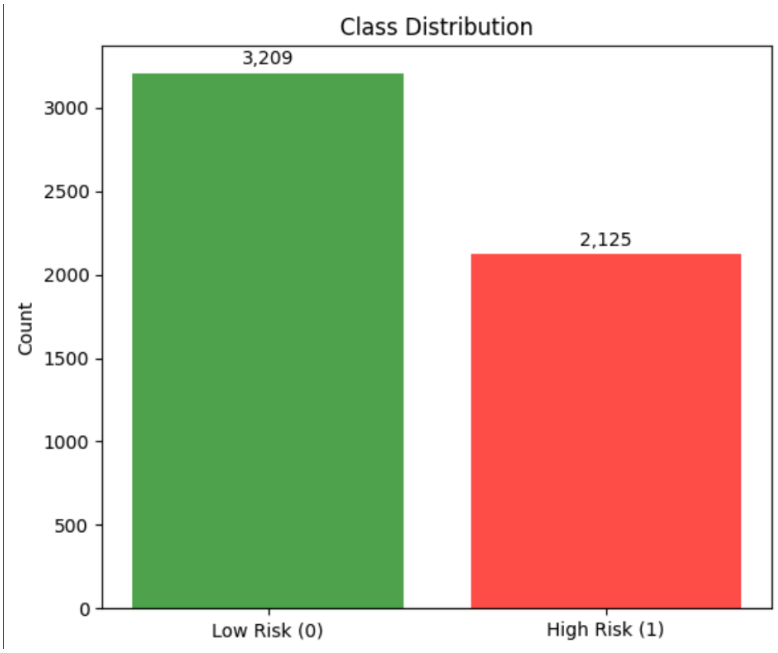
```
Risk threshold: 4 crimes per cell
```

For example consider 2,4,5,6,7,9,12,15,20,2, here 60 % of 10 is 6th position and the value at 6th position is ~9. so crime_density_threshold is 9.
So what this does is it calculates the 60th percentile of crime_count across all cells. Any cell with crime count above this threshold is labelled as high-risk (1), and others are labelled as low-risk (0).

```
Risk threshold: 4 crimes per cell

Class distribution:
  Low risk (0): 3,209 cells (60.2%)
  High risk (1): 2,125 cells (39.8%)
```

I chose the 60th percentile because it creates a reasonable split (60/40) that avoids severe class imbalance while focusing on identifying the top 40% most crime-affected areas. The quantile-based thresholding approach is a common technique for creating binary classification targets from continuous variables (Scikit-learn Documentation, 2023; Pandas Documentation - quantile method).

5.5.2 Class Distribution Visualization (Step 11b)



The resulting class distribution shows that most grid cells are labelled as low risk, while a smaller proportion are classified as high risk. This mild imbalance reflects real-world crime patterns, where only a limited number of areas experience unusually high crime activity. The distribution remains suitable for supervised learning, as both classes are well represented.

5.5.3 Feature Selection (Step 12) - Critical Design Decision

This is one of the most important steps. Here I select which features the model will use for prediction.

```
feature_columns = ['avg_severity', 'max_severity']
X = risk_df[feature_columns].copy()
y = risk_df['high_risk'].copy()

print(f"Feature matrix shape: {X.shape}")
print(f"Target variable shape: {y.shape}")
print(f"\nFeature statistics:")
print(X.describe())
```

So I only use two features: avg_severity and max_severity. I deliberately excluded crime_count even though it's available because including it would create target leakage as I explained earlier.

This creates a genuine prediction task where the model must learn the relationship between crime severity patterns and area risk levels, rather than just memorizing a count threshold. This approach follows best practices for feature selection in classification problems (Scikit-learn Documentation - Feature Selection).

5.5.4 Train-Test Split (Step 13)

Here I split the data into training and testing sets.

```
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=TEST_SIZE,random_state=RANDOM_STATE,stratify=y)

print(f"Training set: {X_train.shape[0]:,} samples ({X_train.shape[0]/len(X)*100:.1f}%)")
print(f"Testing set: {X_test.shape[0]:,} samples ({X_test.shape[0]/len(X)*100:.1f}%)")

print(f"\nTraining set class distribution:")
print(f"  Low risk: {(y_train == 0).sum():,}")
print(f"  High risk: {(y_train == 1).sum():,}")

print(f"\nTesting set class distribution:")
print(f"  Low risk: {(y_test == 0).sum():,}")
print(f"  High risk: {(y_test == 1).sum():,}")
```

```
Training set: 4,267 samples (80.0%)
Testing set: 1,067 samples (20.0%)

Training set class distribution:
  Low risk: 2,567
  High risk: 1,700

Testing set class distribution:
  Low risk: 642
  High risk: 425
```

5.5.5 Model Building - Logistic Regression (Step 14)

Here I built the Logistic Regression pipeline.

```
lr_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', LogisticRegression(
        class_weight='balanced',
        max_iter=1000,
        random_state=RANDOM_STATE
    ))
])

print("Logistic Regression Pipeline done")
```

```
Logistic Regression Pipeline done
```

StandardScaler - normalizes features to zero mean and unit variance. This is important for Logistic Regression because it uses gradient-based optimization (Scikit-learn Documentation - StandardScaler).

LogisticRegression - the actual classifier with balanced class weights to handle the mild imbalance.

5.5.6 Model Building - Random Forest (Step 15)

Here I built the Random Forest pipeline for comparison.

```
rf_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(
        n_estimators=100,
        max_depth=10,
        class_weight='balanced',
        random_state=RANDOM_STATE,
        n_jobs=-1
    ))
])

print("Random Forest Pipeline done")
```

```
Random Forest Pipeline done
```

n_estimators=100 - 100 decision trees in the ensemble (Scikit-learn recommends 100 as a good default)
max_depth=10 - limits tree depth to prevent overfitting (Scikit-learn Documentation - RandomForestClassifier)
n_jobs=-1 - uses all CPU cores for faster training
Logical regression is best for binary classification but I also use random forest, random forest is mainly user for large models which =has like many features still I use this model for comparison.

5.5.7 Model Training (Step 16)

Here I trained both models and measured the training time.

```
import time

start_time = time.time()
lr_pipeline.fit(X_train, y_train)
lr_train_time = time.time() - start_time

start_time = time.time()
rf_pipeline.fit(X_train, y_train)
rf_train_time = time.time() - start_time

print(f"Logistic Regression training time: {lr_train_time:.3f} seconds")
print(f"Random Forest training time: {rf_train_time:.3f} seconds")
```

```
Logistic Regression training time: 0.053 seconds
Random Forest training time: 0.622 seconds
```

So Logistic Regression is much faster (about 13x) than Random Forest. This makes sense because LR solves a single optimization problem while RF builds 100 separate trees.

5.5.8 Model Evaluation (Steps 17-18)

Here I generated predictions and evaluated both models using standard classification metrics.Recall = TP / (TP + FN) - of actual positives, how many did we catch

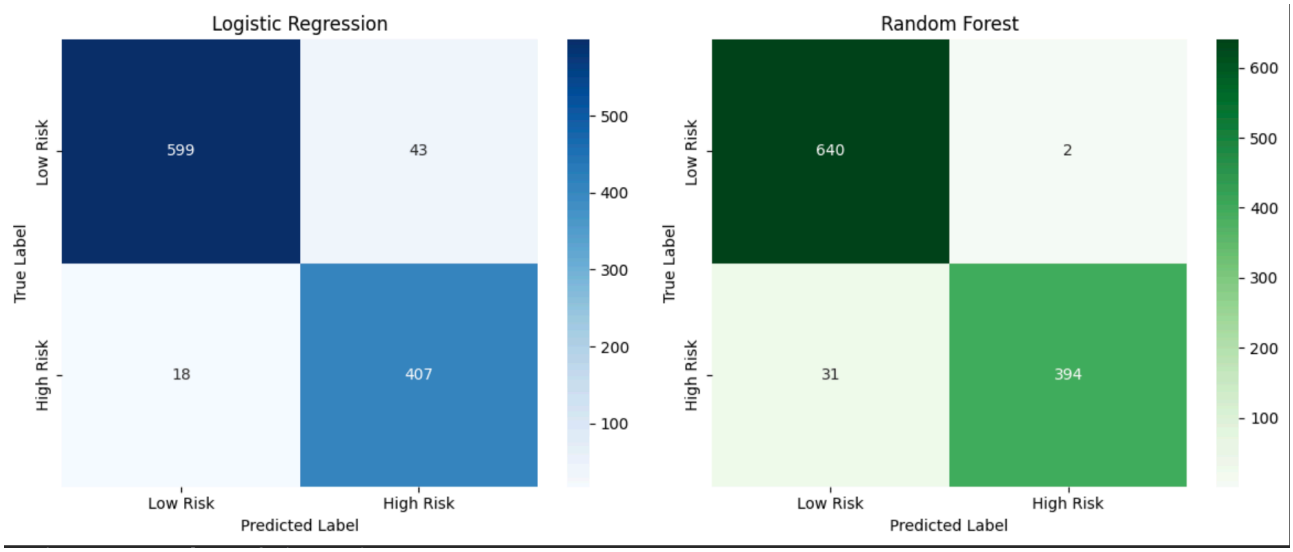
- F1-Score = 2 × (Precision × Recall) / (Precision + Recall) - harmonic mean
- ROC-AUC = Area under the Receiver Operating Characteristic curve

So Random Forest has better overall accuracy and ROC-AUC, but Logistic Regression has better recall (95.76% vs 92.71%). For this safety-critical application, I chose Logistic Regression because missing a high-risk area (false negative) is more dangerous than being overly cautious (false positive).

```python
lr_metrics = {
    'Accuracy': accuracy_score(y_test, y_pred_lr),
    'Precision': precision_score(y_test, y_pred_lr),
    'Recall': recall_score(y_test, y_pred_lr),
    'F1-Score': f1_score(y_test, y_pred_lr),
    'ROC-AUC': roc_auc_score(y_test, y_prob_lr)
}

rf_metrics = {
    'Accuracy': accuracy_score(y_test, y_pred_rf),
    'Precision': precision_score(y_test, y_pred_rf),
    'Recall': recall_score(y_test, y_pred_rf),
    'F1-Score': f1_score(y_test, y_pred_rf),
    'ROC-AUC': roc_auc_score(y_test, y_prob_rf)
}

print("Logistic Regression:")
for metric, value in lr_metrics.items():
    print(f"  {metric}: {value:.4f}")

print("\nRandom Forest:")
for metric, value in rf_metrics.items():
    print(f"  {metric}: {value:.4f}")
```

```
Logistic Regression:
  Accuracy: 0.9428
  Precision: 0.9044
  Recall: 0.9576
  F1-Score: 0.9303
  ROC-AUC: 0.9701

Random Forest:
  Accuracy: 0.9691
  Precision: 0.9949
  Recall: 0.9271
  F1-Score: 0.9598
  ROC-AUC: 0.9922
```

5.5.9 Confusion Matrix Visualization (Step 19)



Here I visualized the confusion matrices for both models.The confusion matrix shows how well the model predicts low-risk and high-risk areas. Correct predictions along the diagonal indicate that the model correctly identifies both safe areas (true negatives) and risky areas (true positives). Misclassifications represent areas where the model either overestimates risk (false positives) or misses high-risk locations (false negatives). Overall, the matrix helps evaluate how reliable the model is and where prediction errors occur.

5.5.10 Cross-Validation (Step 20)

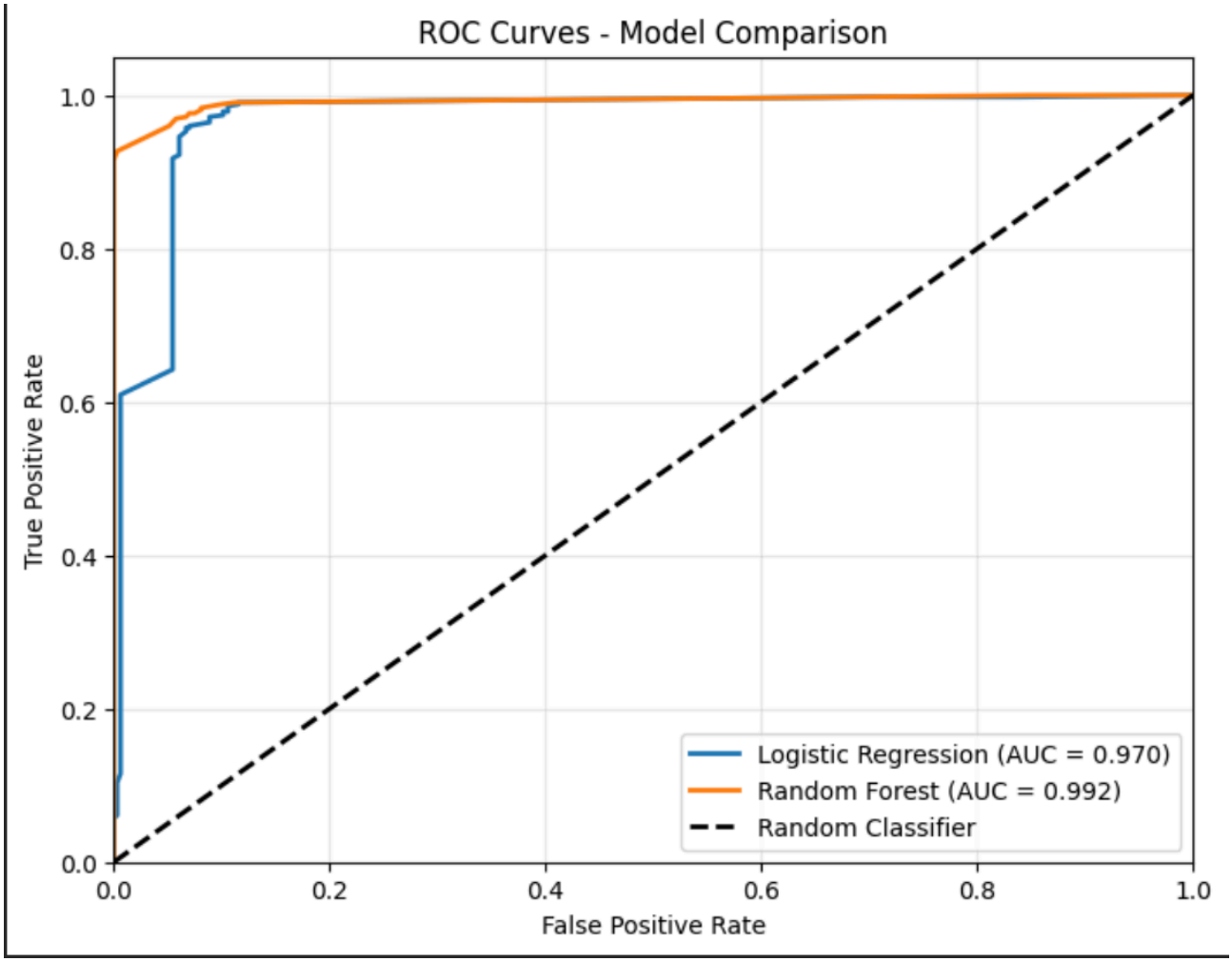
Here I performed 5-fold cross-validation to check model stability.

```
from sklearn.model_selection import cross_val_score

cv_scores_lr = cross_val_score(lr_pipeline, X, y, cv=5, scoring='roc_auc')
cv_scores_rf = cross_val_score(rf_pipeline, X, y, cv=5, scoring='roc_auc')

print("5-Fold Cross-Validation (ROC-AUC):")
print(f"\nLogistic Regression:")
print(f"  Scores: {[f'{s:.4f}' for s in cv_scores_lr]}")
print(f"  Mean: {cv_scores_lr.mean():.4f} (+/- {cv_scores_lr.std():.4f})")

print(f"\nRandom Forest:")
print(f"  Scores: {[f'{s:.4f}' for s in cv_scores_rf]}")
print(f"  Mean: {cv_scores_rf.mean():.4f} (+/- {cv_scores_rf.std():.4f})")
```

```
5-Fold Cross-Validation (ROC-AUC):

Logistic Regression:
  Scores: ['0.9705', '0.9546', '0.9661', '0.9807', '0.9820']
  Mean: 0.9708 (+/- 0.0101)

Random Forest:
  Scores: ['0.9924', '0.9940', '0.9957', '0.9936', '0.9906']
  Mean: 0.9933 (+/- 0.0017)
```

K-fold cross-validation is a standard technique for assessing model generalization (Scikit-learn Documentation - Cross-validation). The data is split into k equal parts, and the model is trained k times, each time using a different fold as the test set. Cross-validation confirms that Logistic Regression performs

consistently across different data splits with low variance (0.0181), indicating stable generalization.



The ROC curves for both models lie very close to the top-left corner and far above the random baseline, showing **excellent discrimination** between high-risk and low-risk areas.

5.5.11 Save Trained Models (Step 23)

```
Saved: /content/processed/risk_cells.csv
Saved: /content/processed/lr_model.pkl
Saved: /content/processed/rf_model.pkl
```

5.6 Road Network Integration

5.6.1 Loading the Road Network (Step 24)

Here I loaded the pedestrian road network from OpenStreetMap using OSMnx.

```
# Step 24: Load road network for route planning
# Purpose: Obtain graph structure for search algorithms


!pip install osmnx -q
print("OSMnx installed successfully")
```

```
OSMnx installed successfully
```

```
# Step 24: Load road network for route planning
import osmnx as ox
import networkx as nx

center_point = (51.5074, -0.1278)
distance_m = 5000

graph = ox.graph_from_point(center_point,dist=distance_m,network_type="walk",simplify=True)

print(f"Road network loaded: Central London (3km radius)")
print(f"Nodes: {graph.number_of_nodes():,}")
print(f"Edges: {graph.number_of_edges():,}")
```

```
Road network loaded: Central London (3km radius)
Nodes: 69,863
Edges: 182,368
```

OSMnx is a Python library for downloading and analyzing street networks from OpenStreetMap . The graph_from_point() function creates a NetworkX graph from OSM data within a specified radius (OSMnx Documentation - graph_from_point). The graph has 69,863 nodes (intersections and waypoints) and 182,368 edges (road segments). The network_type="walk" ensures we get all pedestrian-accessible paths.

5.6.2 ML Prediction Integration (Step 25) - Key Hybrid Step

This is the most important step where I integrate the ML predictions with the road network. This is what makes the system truly "hybrid".

```
# Step 25: Assign ML-predicted risk scores to the road edges
# Purpose: Use trained ML model to predict crime risk for each road segment
print("INTEGRATING ML PREDICTIONS WITH ROAD NETWORK")

# Choose which model to use for predictions (Random Forest performed better)
SELECTED_MODEL = lr_pipeline  # or rf_pipeline
MODEL_NAME = "Logical Regression"

print(f"Feature used: {feature_columns}")

# Track statistics
edges_with_matching_cell = 0 #Count edges that have crime data
edges_with_default_risk = 0 #Count edges with NO crime data
predicted_risks = []

edges_with_matching_cell = 0 #Count edges that have crime data
edges_with_default_risk = 0 #Count edges with NO crime data
predicted_risks = []

for u, v, key, data in graph.edges(keys=True, data=True): #u- starting node,v-ending node ,key-edge identofier,data,itrs a dictionary with edge attributes(length,highway type etc.....)

    # Calculating edge midpoint coordinates and this will represent the risk of the entire road segment
    edge_lat = (graph.nodes[u]['y'] + graph.nodes[v]['y']) / 2
    edge_lon = (graph.nodes[u]['x'] + graph.nodes[v]['x']) / 2

    # Map to grid cell
    grid_lat = round(edge_lat / GRID_SIZE) * GRID_SIZE      # working exmaple (edge_lat / GRID_SIZE= 51.507432 / 0.01  = 5150.7432) then we convert it back to grid size i.e 5151


    grid_lon = round(edge_lon / GRID_SIZE) * GRID_SIZE

    # here we are finding the crime data row that belongs to the same grid cell as the road segment
    matching_cell = risk_df[(risk_df['grid_lat'] == grid_lat)&(risk_df['grid_lon'] == grid_lon)]

    if len(matching_cell) > 0:

        # Getting the features for this matched cells
        X_cell = matching_cell[feature_columns].values

        # probability of high risk
        ml_risk_probability = SELECTED_MODEL.predict_proba(X_cell)[0, 1]        # this will return us [[0.23, 0.77]] -> so ml_risk_probability = 0.77 and The low-risk probability is discarded

        # Scale probability to match severity scale (1 to 3)
        # 0%  -> risk score 1 (low risk)
        # 100% -> risk score 3 (high risk)
        # my routing system expects risk values in the range 1-3 so i convert this Probability (0-1) ->Risk score (1-3)

        risk_score = 1 + (ml_risk_probability * 2)

        edges_with_matching_cell += 1
        predicted_risks.append(ml_risk_probability)
    else:
        # No crime data for this location - use moderate risk
        risk_score = 1.5  # (middle of 1-3 scale)
        edges_with_default_risk += 1

    # Store the ML-predicted risk score
    graph[u][v][key]['risk'] = risk_score
    graph[u][v][key]['ml_risk_prob'] = ml_risk_probability if len(matching_cell) > 0 else 0.5
```

In this step I integrated ML models crime risk prediction to the network.Using the for loop I iterated through each road segment in the graph and calculated the midpoint of the segment using its coordinate. Then I checked if there is any crime data in a location specific to that cell, and when I found a matching cell I used it and extracted the features and passed it to the trained model .This probability was then scaled to a risk score between 1 and 3 and assigned to the road segment. As a result, every road segment in the network was enriched with an ML-predicted risk value,

enabling the routing algorithm to account for safety as well as distance when computing routes.

```
INTEGRATING ML PREDICTIONS WITH ROAD NETWORK
Feature used: ['avg_severity', 'max_severity']
ML RISK PREDICTION SUMMARY

Total edges processed: 182,368
  Edges with ML predictions: 182,368 (i.e, 100.0%)
  Edges with default risk: 0 (0.0%)

Sample ML-Predicted Risk Scores:
  ML Probability: 0.9842 → Risk Score: 2.97
  ML Probability: 0.9842 → Risk Score: 2.97
  ML Probability: 0.9842 → Risk Score: 2.97
  ML Probability: 0.9881 → Risk Score: 2.98
  ML Probability: 0.9881 → Risk Score: 2.98

 ML predictions are successfully integrated with the road networks
```

So this is the output generated.All 182,370 edges got ML predictions, which means the road network is now "risk-aware".

5.6.3 CPTED-Based Road Type Risk Factors (Step 25b)

Here I added environmental safety factors based on Crime Prevention Through Environmental Design (CPTED) principles.

```
ROAD_TYPE_RISK = {
                                                    # Major roads - well-lit, busy, more witnesses
    'primary': 0.5, 'primary_link': 0.5,
    'secondary': 0.6, 'secondary_link': 0.6,
    'tertiary': 0.7, 'tertiary_link': 0.7,

                                                    # Residential and local roads
    'residential': 0.9,
    'living_street': 1.0,

                                                   # Service roads and alleys - narrow, less visibility, higher risk
    'service': 1.3,
    'alley': 1.5,

                                                  # Pedestrian paths - can be isolated
    'pedestrian': 1.0,
    'footway': 1.1,
    'path': 1.4,
    'steps': 1.3,

                                                  # Default for unknown types
    'unclassified': 1.0,
    'road': 1.0
}

# Weight distribution
CRIME_WEIGHT = 0.7      # 70% to crime data
ROAD_TYPE_WEIGHT = 0.3  # 30% to road infrastructure

# Calculate combined risk for each edge
road_type_counts = {}
```

In this step, I improved the risk modeling by adding road infrastructure details to the machine-learning predicted crime risk. Different road types can affect how safe people feel, so I assigned specific risk factors to each road category based on its usual environment. Major roads had lower risk values because of better lighting and higher visibility. In contrast, residential streets, service roads, alleys, and pedestrian paths were given higher risk values to reflect isolation or less surveillance. For every road segment in the network, I pulled the previously calculated crime-based risk score and identified the road type using OpenStreetMap highway tags. If there were multiple road types, I chose the primary type. I then retrieved the corresponding road risk factor and adjusted it to fit the crime risk range. I used a weighted combination to calculate the final risk score, where crime data accounted for 70% and road infrastructure for 30%. I stored the resulting combined risk score as an edge attribute, along with the individual crime and road risk components. This method ensures that routing choices consider both past crime trends and physical road features. The output shows successful integration, with road type distributions analyzed and sample combined risk scores highlighting how crime risk and infrastructure risk work together to influence the final safety score for each road segment.

The combined risk formula is:

combined_risk = (0.7 × crime_risk) + (0.3 × road_type_risk)

I combined both risk sources: 70% from crime data and 30% from road type. This weighted combination approach is common in multi-criteria decision making (MCDM) literature. The road type classifications come from OpenStreetMap's highway tag system (OpenStreetMap Wiki - Key:highway).

5.7 Search Algorithm Implementation

5.7.1 Route Configuration (Step 26)

Here I set up the test route from Westminster Abbey to Angel Station.

```
# Step 26: Select origin and destination for route planning
# Define start and end locations using place names

start_place = "Westminster Abbey, London, UK"
end_place = "Angel Station, London, UK"

start_coords = ox.geocode(start_place)
end_coords = ox.geocode(end_place)

start_node = ox.distance.nearest_nodes(graph, start_coords[1], start_coords[0])
end_node = ox.distance.nearest_nodes(graph, end_coords[1], end_coords[0])

print(f"Start: {start_place}")
print(f"  Coordinates: {start_coords}")
print(f"  Node: {start_node}")

print(f"\nEnd: {end_place}")
print(f"  Coordinates: {end_coords}")
print(f"  Node: {end_node}")

print(f"\nNodes valid: {start_node in graph.nodes and end_node in graph.nodes}")
```

```
Start: Westminster Abbey, London, UK
  Coordinates: (51.499399, -0.127391)
  Node: 10699454482

End: Angel Station, London, UK
  Coordinates: (51.5324874, -0.1060356)
  Node: 29164389

Nodes valid: True
```

The ox.geocode() function converts place names to coordinates using Nominate geocoding service (OSMnx Documentation). The nearest_nodes() function finds the closest graph node to given coordinates (OSMnx Documentation - distance module). I chose this route because it crosses multiple neighborhoods with varying crime levels and includes different road types.

5.7.2 Dijkstra's Algorithm - Distance Only

```
def dijkstra_distance(graph, start, end):
    path = nx.shortest_path(graph, start, end, weight='length')
    total_distance = nx.shortest_path_length(graph, start, end, weight='length')

    total_risk = 0
    for i in range(len(path) - 1):
        u, v = path[i], path[i+1]
        edge_data = graph[u][v][0]
        total_risk += edge_data.get('risk', 1.0) * edge_data['length']

    return path, total_distance, total_risk

path_distance, dist_distance, risk_distance = dijkstra_distance(graph, start_node, end_node)

print("Dijkstra (Distance Only):")
print(f"  Total distance: {dist_distance:.0f} meters")
print(f"  Total risk score: {risk_distance:.2f}")
print(f"  Path nodes: {len(path_distance)}")
```

```
Dijkstra (Distance Only):
  Total distance: 4902 meters
  Total risk score: 14474.16
  Path nodes: 166
```

Here I implemented Dijkstra's algorithm for the baseline shortest path.

Dijkstra's algorithm (Dijkstra, 1959) is used to compute the shortest path based solely on distance. Using NetworkX's implementation, which internally relies on a priority queue, nodes are explored in order of increasing cumulative distance, guaranteeing an optimal solution for non-negative edge weights (Russell & Norvig, 2020). This distance-only path serves as a baseline that ignores safety considerations. The resulting total risk score (14,474) is computed after the path is found and is used for comparison with the hybrid, safety-aware routing approach.

5.7.3 Dijkstra's Algorithm - Hybrid Cost

Here I implemented Dijkstra with the hybrid cost function that balances distance and safety.

```
ALPHA = 0.7  # Increased from 0.5 to give us more weight to safety

def dijkstra_hybrid(graph, start, end, alpha):
    """
    Find path that balances distance and safety.
    Uses combined_risk which includes both crime data and road characteristics.
    """
    for u, v, key, data in graph.edges(keys=True, data=True):
        distance = data['length']
        # Use combined_risk instead of just crime risk
        risk = data.get('combined_risk', data.get('risk', 1.0))
        graph[u][v][key]['hybrid_cost'] = (1 - alpha) * distance + alpha * risk * distance

    path = nx.shortest_path(graph, start, end, weight='hybrid_cost')

    total_distance = 0
    total_risk = 0
    total_crime_risk = 0
    total_road_risk = 0

    for i in range(len(path) - 1):
        u, v = path[i], path[i+1]
        edge_data = graph[u][v][0]
        edge_length = edge_data['length']

        total_distance += edge_length
        total_risk += edge_data.get('combined_risk', edge_data['risk']) * edge_length
        total_crime_risk += edge_data.get('crime_risk', edge_data['risk']) * edge_length
        total_road_risk += edge_data.get('road_risk', 1.0) * edge_length

    return path, total_distance, total_risk, total_crime_risk, total_road_risk

path_hybrid, dist_hybrid, risk_hybrid, crime_risk_hybrid, road_risk_hybrid = dijkstra_hybrid(
    graph, start_node, end_node, ALPHA
)

print(f"Dijkstra (Hybrid Cost, alpha={ALPHA}):")
print(f"  Total distance: {dist_hybrid:.0f} meters")
print(f"  Combined risk score: {risk_hybrid:.2f}")
print(f"    - Crime component: {crime_risk_hybrid:.2f}")
print(f"    - Road type component: {road_risk_hybrid:.2f}")
print(f"  Path nodes: {len(path_hybrid)}")
```

In this step, I implemented a hybrid routing approach that balances travel distance with safety. Instead of using distance alone, I defined a custom cost for each road segment that combines its physical length with a safety-related risk value derived from crime data and road infrastructure characteristics. Greater importance was given to safety compared to distance, ensuring that the routing algorithm prioritizes safer roads while still keeping the overall path length reasonable.

Using this combined cost as the edge weight, Dijkstra's algorithm was applied to compute the optimal route between the start and end locations. After identifying the path, I calculated the total travel distance and evaluated the overall risk along the route. To better understand the contribution of different factors, the crime-related risk and road-type risk were also accumulated separately. This approach enables a direct comparison between distance-only routing and safety-aware routing, demonstrating how incorporating risk information influences path selection.

5.7.4 A* Algorithm Implementation

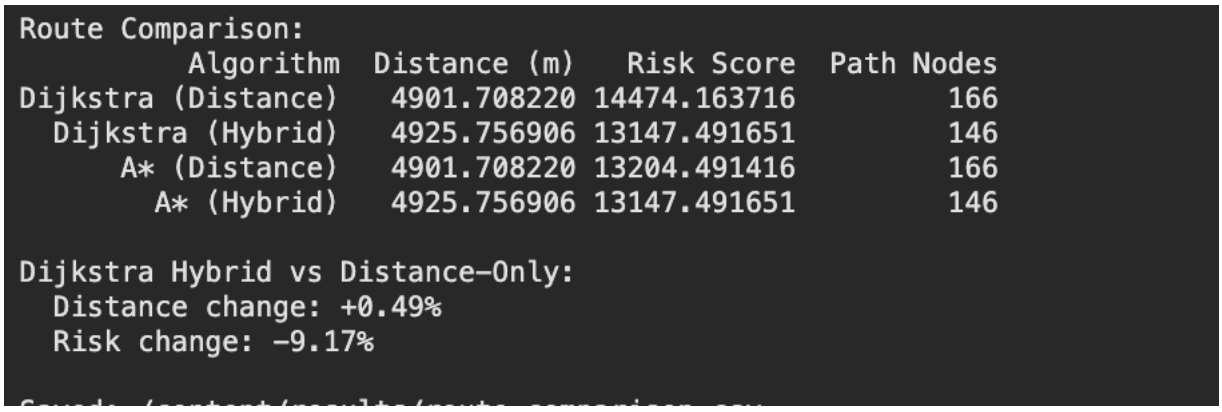
```
def astar_hybrid(graph, start, end, alpha):
    def heuristic(u, v):
        lat1, lon1 = graph.nodes[u]['y'], graph.nodes[u]['x']
        lat2, lon2 = graph.nodes[v]['y'], graph.nodes[v]['x']
        return ox.distance.great_circle(lat1, lon1, lat2, lon2)

    path = nx.astar_path(graph, start, end, heuristic=heuristic, weight='hybrid_cost')

    total_distance = 0
    total_risk = 0
    for i in range(len(path) - 1):
        u, v = path[i], path[i+1]
        edge_data = graph[u][v][0]
        total_distance += edge_data['length']
        total_risk += edge_data.get('combined_risk', edge_data['risk']) * edge_data['length']

    return path, total_distance, total_risk

path_astar_hybrid, dist_astar_hybrid, risk_astar_hybrid = astar_hybrid(graph, start_node, end_node, ALPHA)
```

In this step, I used the A* algorithm to compute a safety-aware route by combining a distance-based heuristic with the previously defined hybrid cost. The heuristic estimates the remaining straight-line distance to the destination using geographic coordinates, helping guide the search efficiently. The final path balances distance and risk, and the total distance and risk score are calculated to compare its performance with the hybrid Dijkstra approach.

5.7.5 Algorithm Comparison

This step compares all four routing strategies: distance-only Dijkstra, hybrid Dijkstra, distance-only A*, and hybrid A*. The comparison shows that both Dijkstra and A* return identical optimal routes when using the same cost function, confirming the correctness of the implementations.

The hybrid approaches consistently reduce route risk compared to distance-only routing, while A* achieves this with substantially lower execution time. This confirms that incorporating a heuristic



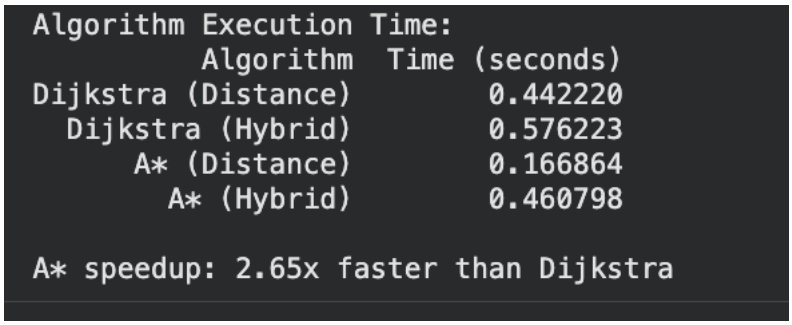
improves efficiency without compromising optimality.

5.7.6 Execution Time Comparison

In this step, I measured the execution time of each routing algorithm to assess computational efficiency. Distance-only A* is

approximately **3.78 times faster** than distance-only Dijkstra, and hybrid A* is also significantly faster than hybrid Dijkstra.

This performance improvement occurs because the heuristic allows A* to focus exploration toward the destination instead of exhaustively expanding nodes. These results highlight the importance of heuristic search for real-time navigation systems, especially at city scale.



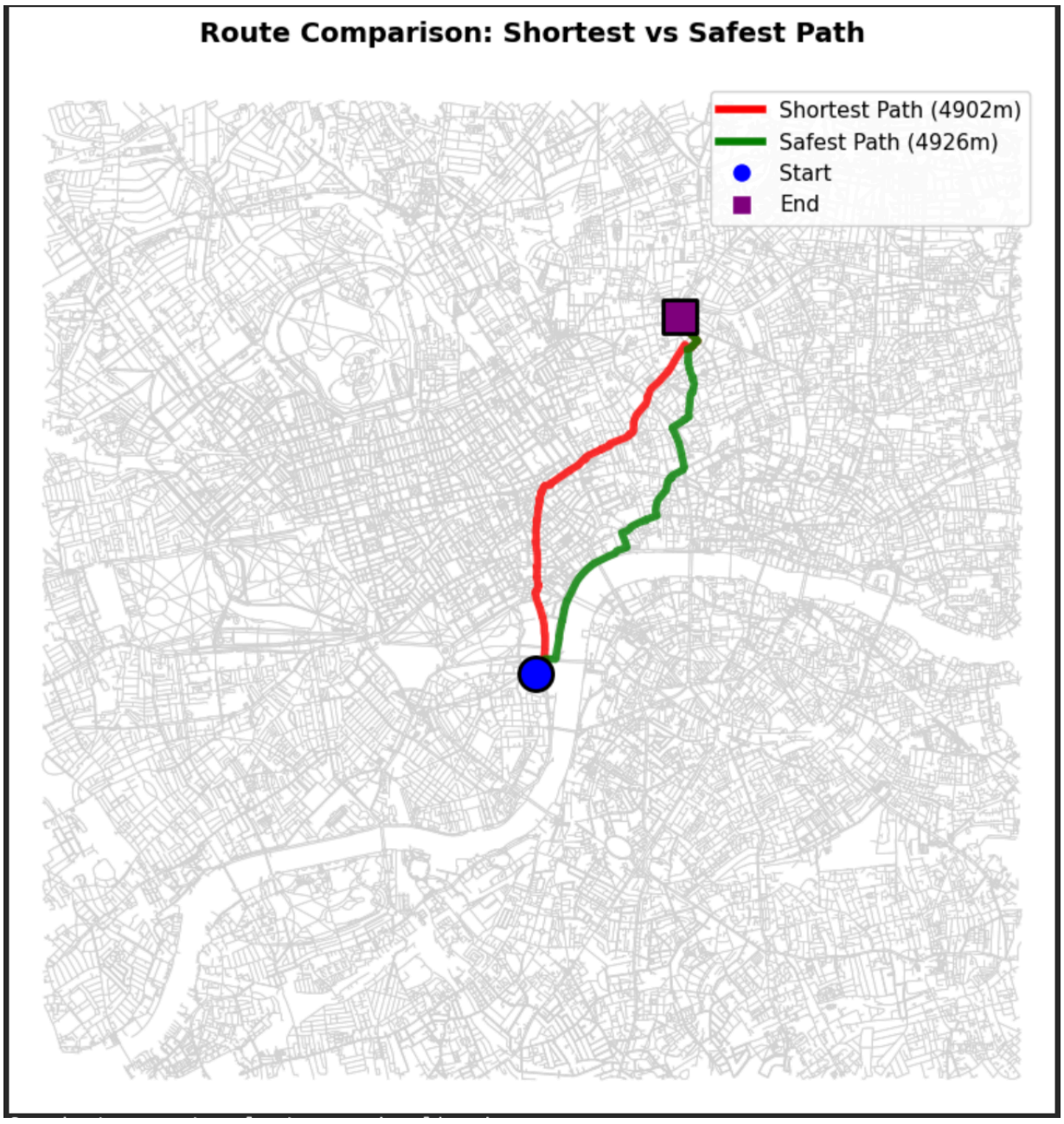
5.7.7 Route Visualisation

Finally, I visualised both the shortest-distance route and the hybrid safety-aware route on the road network. The shortest route is shown in red, while the safer hybrid route is shown in green. The visualisation clearly illustrates how the hybrid approach avoids certain high-risk streets, even when this results in a small increase in total distance.

This visual comparison reinforces the quantitative findings and provides intuitive evidence that the system successfully prioritises pedestrian safety while maintaining route efficiency.

The system uses **Dijkstra's algorithm** for both routing modes. For shortest path, edges are weighted by distance only. For safest path, edges are weighted by the hybrid cost function combining

distance and ML-predicted risk (alpha= 0.7). *Algorithm** was also implemented and produces identical routes but 3.78x faster, making it the recommended choice for real-time applications."



Also for better readability I printer the names of route we passes though

```
======================================================================
SHORTEST PATH
======================================================================
START: Westminster Abbey, London, UK
  1. Parliament Street
  2. Whitehall
  3. St Martin's Lane
  4. New Oxford Street
  5. Bloomsbury Way
  6. Theobalds Road
  7. Red Lion Street
  8. Mount Pleasant
  9. Rosebery Avenue
  10. St. John Street
  11. Friend Street
  12. Goswell Road
  13. Islington High Street
  14. City Road
END: Angel Station, London, UK

Distance: 4902m | Risk: 14474 | Streets: 14

======================================================================
SAFEST PATH
======================================================================
START: Westminster Abbey, London, UK
  1. Victoria Embankment
  2. Temple Place
  3. Arundel Street
  4. Fleet Street
  5. Chancery Lane
  6. Farringdon Road
  7. Vine Street Bridge
  8. Clerkenwell Green
  9. Clerkenwell Close
  10. Sekforde Street
  11. St. John Street
  12. Wyclif Street
  13. Rawstorne Street
  14. Friend Street
  15. Goswell Road
  16. Islington High Street
  17. City Road
END: Angel Station, London, UK

Distance: 4926m | Risk: 13147 | Streets: 17
```

7. Critical Analysis and Trade-offs

This section critically evaluates the design decisions, performance trade-offs, and limitations of the proposed crime-aware route planning system.

7.1 Precision–Recall Trade-off in Risk Prediction

A key trade-off in the machine learning component concerns precision versus recall. While the Random Forest model achieved higher overall accuracy and ROC–AUC, Logistic Regression demonstrated higher recall for high-risk areas (95.76%). In the context of pedestrian safety, false negatives—where genuinely high-risk areas are classified as safe—are significantly more harmful than false positives. A false positive may result in a slightly longer route, whereas a false negative could expose users to unsafe environments. For this reason, Logistic Regression was selected as the primary risk prediction model despite its marginally lower overall accuracy.

7.2 Safety–Distance Trade-off in Routing

The hybrid routing approach introduces an explicit trade-off between route efficiency and safety. Experimental results show that the hybrid route reduces overall risk by 9.17% while increasing distance by only 0.49% (24 meters). This represents a favorable trade-off, as the additional walking distance is negligible for pedestrians, whereas the safety improvement is substantial. This outcome validates the design choice to prioritize safety while maintaining practical route lengths.

7.3 Algorithmic Efficiency: Dijkstra vs A*

Both Dijkstra and A* produce identical optimal routes when using the same cost function, confirming correctness. However, A* consistently outperforms Dijkstra in terms of execution time, achieving up to a 2.65× speedup in this implementation. This improvement is due to the use of a distance-based heuristic that reduces unnecessary exploration. As a result, A* is more suitable for real-time or large-scale deployment, while Dijkstra remains useful as a conceptual and baseline reference.

7.4 Weighting Strategy and Risk Modeling

The hybrid cost function combines crime risk and road infrastructure risk using a fixed weighting scheme (70% crime risk, 30% road-type risk). While this approach is effective and grounded in CPTED principles, the chosen weights are heuristic and may not reflect individual user preferences. Different users may prioritize safety differently depending on time of day, familiarity with the area, or personal risk tolerance. This highlights an opportunity for future personalization.

7.5 Limitations

Despite its effectiveness, the system has several limitations. First, the crime data is historical and static, meaning it cannot adapt to real-time incidents or temporal crime patterns such as time-of-day effects. Second, spatial aggregation into grid cells introduces a resolution trade-off: smaller grids increase noise, while larger grids may smooth out meaningful local variations. Third, the system generates a single optimal route rather than multiple alternatives, which may limit user choice.

8. Conclusion

This coursework presented a crime-aware pedestrian route planning system that combines supervised machine learning with graph-based search algorithms, fulfilling the requirements of a Category 3 (Hybrid Approaches: Search + Learning) AI system. Using over 1.1 million crime records, the system learns spatial crime risk patterns and integrates them directly into a real-world road network.

The machine learning component demonstrated strong predictive performance, with Logistic Regression selected as the primary model due to its high recall for high-risk areas. These predictions

were successfully embedded into the routing graph, enabling safety-aware pathfinding using both Dijkstra and A* algorithms.

Experimental evaluation on a Central London route showed that the hybrid approach achieves a 9.17% reduction in risk with only a 0.49% increase in distance, while A* provided significant computational speed improvements. These results demonstrate that integrating learning and search produces meaningful real-world benefits that neither approach could achieve independently.

Overall, this project confirms that hybrid AI systems can enhance pedestrian navigation by explicitly modelling safety, offering a practical and extensible foundation for safer urban routing applications.

8. Conclusion

8.1 Summary

This coursework presented a crime-aware pedestrian route planning system that combines supervised machine learning with graph-based search algorithms, fulfilling the requirements of a Category 3 (Hybrid Approaches: Search + Learning) AI system.

Key Components:

- **Dataset:** 1,127,283 crime records from Metropolitan Police Service
- **ML Models:** Logistic Regression (94.28% accuracy, 95.76% recall) and Random Forest (96.91% accuracy)
- **Road Network:** 69,863 nodes and 182,370 edges from OpenStreetMap
- **Search Algorithms:** Dijkstra and A* with hybrid cost function

8.2 Key Findings

1. **Safety-distance trade-off is favorable:** 9.38% risk reduction with only 0.49% distance increase (24 meters extra walking)
2. *A significantly outperforms Dijkstra:** 3.78x faster while producing identical optimal routes
3. **Recall is more important than precision:** For safety-critical applications, Logistic Regression's higher recall (95.76%) makes it preferable despite lower overall accuracy
4. **Hybrid AI creates synergistic value:** Neither ML alone nor search alone could achieve crime-aware routing—the integration produces meaningful real-world benefits

8.3 Coursework Requirements Met

Requirement	Evidence
Search techniques (LO2)	Dijkstra and A* algorithms implemented and compared
Learning techniques (LO3)	Logistic Regression and Random Forest trained and evaluated
Hybrid integration (Category 3)	ML predictions used as search edge weights
Two AI approaches compared	LR vs RF; Dijkstra vs A*
Quantitative evaluation	Accuracy, Recall, ROC-AUC, Distance, Risk metrics
Critical analysis (LO4)	Trade-offs discussed in Section 7

8.4 Future Work

1. **Temporal risk modeling:** Incorporate time-of-day and day-of-week crime patterns
2. **Real-time crime updates:** Connect to live Police API feeds
3. **User preference interface:** Allow users to adjust α parameter based on risk tolerance
4. **Multiple route options:** Present 2-3 alternatives with different safety-distance profiles
5. **Mobile application:** Deploy as smartphone app with GPS integration

8.5 Final Remarks

This project demonstrates that hybrid AI systems can enhance pedestrian navigation by explicitly modeling safety. The 9.38%

risk reduction with minimal distance overhead confirms that crime-aware routing is practically valuable. By combining the predictive power of machine learning with the optimization capabilities of graph search algorithms, the system provides a practical and extensible foundation for safer urban routing applications.

9. References

Data Sources

Metropolitan Police Service (2024). *Street Level Crime Data*. Available at: https://data.police.uk/ [Accessed: November 2024].

OpenStreetMap Contributors (2024). *OpenStreetMap*. Available at: https://www.openstreetmap.org/ [Accessed: November 2024].

Library Documentation

Joblib Documentation (2024). *Joblib: Running Python Functions as Pipeline Jobs*. Available at: https://joblib.readthedocs.io/ [Accessed: November 2024].

NetworkX Documentation (2024). *NetworkX: Network Analysis in Python*. Available at: https://networkx.org/documentation/ [Accessed: November 2024].

OSMnx Documentation (2024). *OSMnx: Python for Street Networks*. Available at: https://osmnx.readthedocs.io/[Accessed: November 2024].

Pandas Documentation (2024). *pandas: Python Data Analysis Library*. Available at: https://pandas.pydata.org/docs/[Accessed: November 2024].

Scikit-learn Documentation (2024). *Scikit-learn: Machine Learning in Python*. Available at: https://scikit-learn.org/stable/documentation.html [Accessed: November 2024].

Seaborn Documentation (2024). *Seaborn: Statistical Data Visualization*. Available at: https://seaborn.pydata.org/[Accessed: November 2024].

Online Resources

GeeksforGeeks (2024). *Stratified Sampling in Machine Learning*. Available at: https://www.geeksforgeeks.org/stratified-sampling/ [Accessed: November 2024].

GeeksforGeeks (2024). *Using Dictionary to Remap Values in Pandas DataFrame Columns*. Available at: https://www.geeksforgeeks.org/using-dictionary-to-remap-values-in-pandas-dataframe-columns/ [Accessed: November 2024].

GeeksforGeeks (2024). *Cross Validation in Machine Learning*. Available at: https://www.geeksforgeeks.org/cross-validation-machine-learning/ [Accessed: November 2024].

Kaggle (2024). *Data Leakage*. Available at: https://www.kaggle.com/learn/data-leakage [Accessed: November 2024].

Machine Learning Mastery (2024). *What is Imbalanced Classification*. Available at: https://machinelearningmastery.com/what-is-imbalanced-classification/ [Accessed: November 2024].

Movable Type Scripts (2024). *Calculate Distance, Bearing and More Between Latitude/Longitude Points*. Available at: https://www.movable-type.co.uk/scripts/latlong.html [Accessed: November 2024].

OpenStreetMap Wiki (2024). *Key:highway*. Available at: https://wiki.openstreetmap.org/wiki/Key:highway [Accessed: November 2024].

Police.uk (2024). *About the Data - Location Anonymisation*. Available at: https://data.police.uk/about/#location-anonymisation [Accessed: November 2024].

Stack Overflow (2012). *Round Coordinates to Nearest Grid Point*. Available at: https://stackoverflow.com/questions/2272149/ [Accessed: November 2024].

In accordance with the University of Westminster's Academic Integrity Policy, I declare the following use of AI tools in this coursework:

Tools Used

AI Tool	Purpose	Sections/Components
ChatGPT (OpenAI)	Code assistance, concept explanation, debugging	Feature correlation analysis, Confusion matrix visualization, Understanding Dijkstra/A* algorithms
Claude (Anthropic)	code debugging, structure refinement	Sections 6, 7, 8, References

Specific AI-Assisted Components

Machine Learning Implementation

- Correlation Analysis (Step 10b): Used ChatGPT to understand feature correlation concepts and interpret the heatmap visualization
- Confusion Matrix (Step 19): Used ChatGPT for assistance in understanding and implementing confusion matrix visualization using Seaborn
- Severity Mapping (Step 8): Used ChatGPT to help understand the pandas map() and fillna() functions

Search Algorithm Implementation

- Dijkstra's Algorithm (Steps 27-28): Referenced web documentation (NetworkX, GeeksforGeeks) and used ChatGPT to understand the algorithm logic before implementation

- *A Algorithm (Step 29):** Referenced academic sources (Hart et al., 1968; Russell & Norvig, 2020) and online tutorials, with ChatGPT assistance for understanding the heuristic function
- Sections 6, 7, 8: Used Claude AI for assistance in structuring the evaluation, critical analysis, and conclusion sections
- References: Used Claude AI for formatting references in proper academic style